Project 1: Parallel Programming

ECE/CSC 406/506, Spring 2026

This project is designed to give you a bit of experience writing and running a shared memory parallel program. Starting with the program specification, you will use OpenMP to write code that will run using 1 - 16 threads, using various work-sharing strategies. You will then collect performance data and write a report discussing your findings.

The assignment will be submitted in three separate stages.

1. Certificate of completion for the HPC Workshop. (10 pts, due 02/27)
2. Initial code that computes the correct result. (50 pts, due 02/27)
3. Final code and performance analysis. (40 pts, due 03/06)

Part 1: HPC Workshop

Complete the training workshop that is offered via REPORTER:

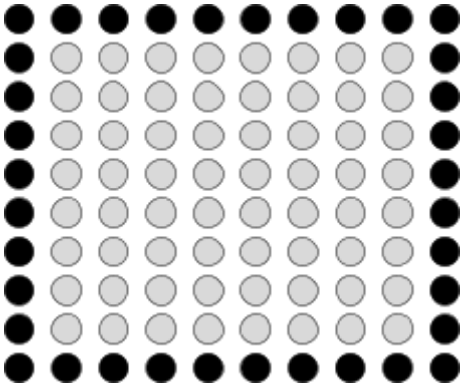
https://reporter.ncsu.edu/link/courseview?courseID=OIT-HPCHen-Basic&deptName=OIT

Submission

You will get a competition certificate by email. Submit that certificate as a PDF file to the **Project 1.1** assignment on the Moodle site.

Part 2: The Program

We will be implementing a very simplified version of a program that iteratively calculates the surface temperature of the ocean, modeled as a 2D grid of points. For this assignment, the grid will always be square, and each side will be 2+N, where N is a power of two.



This example grid is 10x10. The outer points (black) represent the boundary conditions, which do not change during the computation. What we are interested in is the 8x8 square of points in the middle (gray).

At each time stamp, each interior point is updated by calculating the average of the values of itself and its neighbors (up, down, left, right) in the previous time stamp.

Given an initial set of values, your code must calculate the final value of the inner points after K time steps.

The natural way to solve this problem is to create a (2+N)x(2+N) array, in which each element is the value of a given point. Note, however, that each point should not be updated immediately at time step *k*, because its neighbor points depend on the value at time step *k*-1. To solve this, we can use two arrays: one for writing and one for reading. (You can think of this as two separate 2D arrays, or as a 3D array with two layers.)

Your job is to write a C program named **ocean.c** that takes four command-line arguments:

- size of the grid, which is assumed to be N+2 where N is a power of 2
- number of time steps
- name of an input data file, which contains the initial data points for the grid
- name of an output file, which will contain the data for only the interior points of the calculated grid

The size of the output grid is NxN. Do not write the boundary conditions.

The data files contain floating point data in row-major order. For the input, the first N+2 values are for row 0, the next N+2 are for row 1, etc. Each row ends with a linefeed character, but that's not important: just read the correct number of floating-point values, since the spaces and linefeeds will automatically be skipped.

Likewise the output file must contain NxN data points in row-major order. Print the values using three decimal places. (No need to round. Just print the desired number of digits.)

You will be given some starter code, in case you need a refresher for how to:

- read and write files
- allocate and use multi-dimensional arrays
- use command-line arguments
- call timer functions

A sample program, **ocean_start.c**, does not do any computation. It simply reads data from a file and writes it to another file.

You will also be given some sample grid data files, and a sample 8x8 output file (1000 iterations) for testing. Another program, **ocean_create.c**, can be used to create additional test files.

To compile a C program, **foo.c**, creating an executable file **foo**:

```
gcc -o foo foo.c
```

Your code should also print the execution time to standard output. This is not checked, but it will help you when you are gathering performance data. See the example code for suggestions on how to capture execution time. Do not include time spent reading or writing files.

Once you have the serial version of your code working, use OpenMP to parallelize one or more loops. *Do not set the number of threads in the code.* You should use the **OMP_NUM_THREADS** environment variable to control the number of threads when you run the program. This is what we will do in the autograder when your code is submitted.

To compile a program with OpenMP pragmas:

```
gcc -o foo -fopenmp foo.c
```

Submission

The Moodle assignment for **Project 1.2** links you to Gradescope, where you will submit one file to the autograder. The file must be named ocean.c, and it must include OpenMP pragmas to parallelize at least one loop.

The code will be run with OMP_NUM_THREADS=1 and with OMP_NUM_THREADS=4, and the output file will be checked for correctness for two different input grids.

You can submit as many times as you need, up until the due date. Late submission is allowed with the typical penalty (0.4 points per hour). Note that the late penalty for parts 1.2 and 1.3 will be *cumulative*, so it is a good idea to not submit late for this part!

Part 3: Performance

For the performance analysis part of the project, run your OpenMP program with 1, 2, 4, 8, and 16 threads. For each number of threads (> 1), run all of the following work-sharing (scheduling) options:

- static, number of chunks equals the number of threads
- static, chunk size = 32
- dynamic
- [506 students only] square/rectangle-shaped chunks, number of chunks equals the number of threads

For each option, run the program 10 times. Report the mean, standard deviation, and 95% confidence intervals for each.

You should use the HPC cluster to perform these experiments. Choose a particular model number (e.g., Gold6130 or Plat8358) and specify exclusive access.

For this section, the specifics about what to submit and the report format are TBD. This document will be updated well in advance of the deadline.